UNIVERSIDADE FEDERAL DE SANTA CATARINA CENTRO TECNOLÓGICO DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA CIÊNCIA DA COMPUTAÇÃO

INE5411 - Organização de Computadores
Professor: Marcelo Daniel Berejuck
Relatório: Laboratório II
(24/09/2025)

Thayssa Yasmin Alcantara de Godoi
19201619

Elisa Belquis de Assumpção
24202220

1. Estratégia Geral

A estratégia para resolver o problema de cálculo da matriz $C = A \cdot B^T$ foi dividir a tarefa em procedimentos modulares, seguindo as diretrizes do laboratório. A execução principal é gerenciada pelo main, que coordena a leitura do nome do arquivo, a transposição da matriz B e a multiplicação das matrizes, além de imprimir o resultado tanto no console quanto em um arquivo de texto.

O programa foi projetado para:

- Solicitar ao usuário o nome do arquivo de saída através do procedimento PROC_NOME.
- Realizar a transposição da matriz B para B^T em um procedimento separado;
- Calcular o produto $A \cdot B^T$ em um procedimento próprio, que por sua vez chama o procedimento de transposição;
- Salvar o resultado final, a matriz C, em um arquivo de texto com o nome fornecido pelo usuário;
- Imprimir a matriz C no console para verificação.

2. Modo de Armazenamento

As matrizes

A, B, (bt) e C são armazenadas na seção de dados (.data) do programa como arrays de palavras (.word). Cada elemento de uma matriz 3x3 ocupa 4 bytes, totalizando 36 bytes por matriz.

- a e b: Armazenam os valores iniciais fornecidos no enunciado.
- bt e c: São inicializadas com zeros para servir como buffers para os resultados da transposição e da multiplicação, respectivamente.

Adicionalmente, buffers de memória (.space) foram alocados para armazenar o nome do arquivo (nomeArq) e para a conversão de números para strings (bufNum, bufRev).

3. Definição de Argumentos e Valores de Retorno

A passagem de argumentos e valores de retorno segue rigorosamente a convenção do MIPS conforme as instruções do laboratório:

- **Argumentos**: Todos os argumentos são passados através dos registradores $a0 a $a3.
  - No **PROC_NOME**, $a0 e $a1 são usados para passar o endereço da mensagem e do buffer do nome do arquivo, respectivamente.
  - No **PROC_TRANS**, $a0 recebe o endereço da matriz a ser transposta (b) e $a1 recebe o endereço do destino (bt).
  - No **PROC_MUL**, $a0 recebe o endereço de A, $a1 o de B, $a2 o de bt e $a3 o de c.

- **Valores de Retorno**: Os valores de retorno são passados através do registrador $v0.
  - **PROC_NOME**: Retorna em $v0 o endereço do buffer com o nome do arquivo.
  - **PROC_TRANS**: Retorna em $v0 o endereço da matriz transposta (bt).
  - **PROC_MUL**: Retorna em $v0 o endereço da matriz resultante (c).

4. Descrição dos Procedimentos

- **PROC_NOME**: Este procedimento exibe uma mensagem no console solicitando ao usuário que insira um nome de arquivo. Ele lê a entrada do usuário e substitui o caractere de quebra de linha (\n) por um terminador nulo (\0) para garantir que a string seja formatada corretamente para uso posterior em chamadas de sistema para abrir arquivos.
- **PROC_TRANS**: Este é um procedimento **folha**. Ele realiza a transposição da matriz B para bt de forma explícita, sem usar laços, copiando cada elemento individualmente com base em seus offsets de memória. Por exemplo, o elemento B[i][j] é carregado e salvo na posição de BT[j][i].
- **PROC_MUL**: Este é um procedimento **não folha**, pois ele chama o **PROC_TRANS** para obter a matriz transposta de B antes de realizar a multiplicação. O procedimento calcula o produto escalar de cada linha da matriz A com cada coluna da matriz B^T. Utiliza três laços aninhados para percorrer as matrizes e acumular os produtos. Os registradores de "calle-saved" ($s0 a $s3) são salvos na pilha para preservar seus valores durante a execução do procedimento.

5. Conversão de Inteiros para ASCII

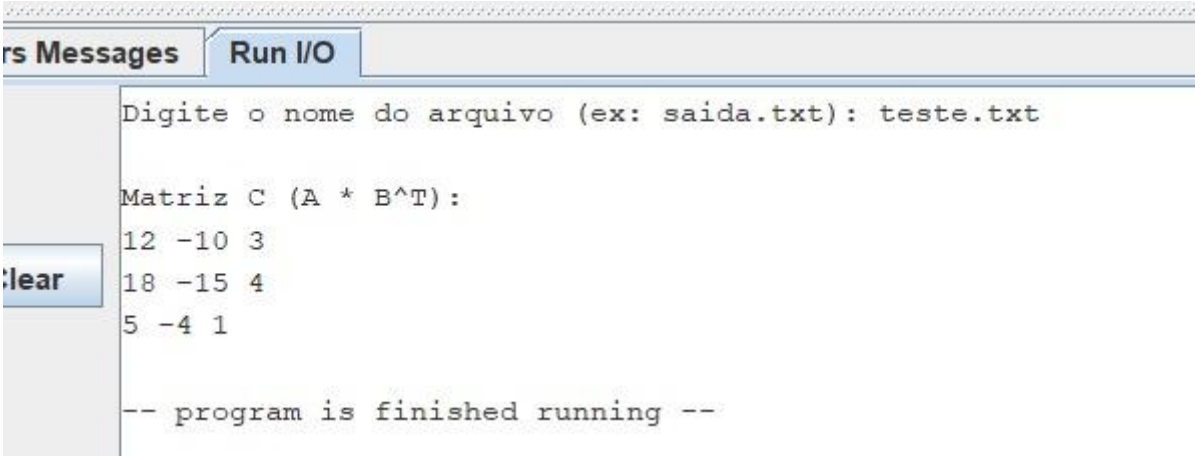
Para cumprir a exigência de salvar os elementos da matriz como texto em um arquivo .txt, foi implementado um procedimento auxiliar chamado **ITOA (Integer to ASCII)**. Este procedimento converte um número inteiro (passado em $a0) para uma string de caracteres ASCII (salva em $a1). A estratégia de conversão envolve:

1. Obter o valor absoluto do número, se ele for negativo.
2. Calcular os dígitos do número usando uma série de divisões por 10.
3. Armazenar os dígitos na ordem inversa em um buffer temporário (bufRev).
4. Copiar os dígitos na ordem correta para o buffer de saída (bufNum).
5. Adicionar o sinal de menos (-) no início da string se o número original for negativo.
6. O comprimento da string convertida é retornado em $v0.

Essa conversão é crucial para que o conteúdo do arquivo de saída seja legível por uma pessoa, em vez de bytes de inteiros.

6. Saída no console do MARS

Quando o programa roda, ele imprime:



Ou seja, cada linha da matriz impressa com os elementos separados por espaço e uma quebra de linha ao final.

7. Conteúdo do arquivo .txt gerado (Em anexo)

Se você digitar por exemplo saida.txt quando o programa pede o nome do arquivo, o arquivo conterá exatamente:

12 -10 3
18 -15 4
5 -4 1

Sem caracteres nulos (\0) ou lixo de memória, apenas os números em ASCII, com espaços entre eles e quebras de linha no final de cada linha.